

Penetration Testing Report: UltraTech (TryHackMe)

Target IP: 10.114.134.99

OS: Linux (Ubuntu)

Objective: Gain root access and retrieve the required flag/artifact.

1. Executive Summary

An authorized security assessment was conducted against the target host 10.114.134.99. The assessment revealed multiple open ports, including an exposed Node.js API endpoint suffering from a Command Injection vulnerability. Exploitation of this vulnerability allowed for database disclosure, leading to the retrieval of administrative credentials. Using these credentials, SSH access was established. Finally, the user escalated privileges to root by leveraging misconfigured Docker group permissions via a known GTFOBins breakout technique.

2. Reconnaissance & Enumeration

2.1 Network Scanning (nmap)

A comprehensive port scan revealed four open services:

Port	State	Service	Version / Framework
21/tcp	Open	FTP	vsftpd 3.0.5
22/tcp	Open	SSH	OpenSSH 8.2p1 Ubuntu 4ubuntu0.13
8081/tcp	Open	HTTP	Node.js Express framework (CORS enabled)
31331/tcp	Open	HTTP	Apache httpd 2.4.41 (Ubuntu)

2.2 Directory Brute-Forcing (gobuster)

Enumeration on the primary web server

(<http://10.114.134.99:31331/>(<http://10.114.134.99:31331/>)) revealed the following paths:

- [/index.html](#) (Status: 200) - Main landing page.
- [/robots.txt](#) (Status: 200) - Basic configuration file.
- [/css/](#), [/images/](#), [/js/](#), [/javascript/](#) (Status: 301) - Static asset directories.

3. Vulnerability Analysis & Exploitation

3.1 Web API Command Injection

While reviewing the JavaScript files located under [/js/](#), an API interaction endpoint was discovered routing requests to a backend ping utility.

- **Vulnerable Endpoint:**
<http://10.114.134.99:31331/ping/ip=127.0.0.1>(<http://10.114.134.99:31331/ping/ip=127.0.0.1>)
- **Vulnerability Type:** Remote Command Injection (RCE) via backticks (`).

By replacing the IP address with a backtick-enclosed shell command, the system executed arbitrary commands on the underlying host.

Step 1: Directory Listing & Database Discovery

Executing [http://10.114.134.99:31331/ping/ip=\]\(http://10.114.134.99:31331/ping/ip=\)ls](http://10.114.134.99:31331/ping/ip=](http://10.114.134.99:31331/ping/ip=)ls) revealed the contents of the directory, including an exposed database file.

Step 2: Data Extraction

Reading the database file via

[http://10.114.134.99:31331/ping/ip=\]\(http://10.114.134.99:31331/ping/ip=\)cat \[database-name\]](http://10.114.134.99:31331/ping/ip=](http://10.114.134.99:31331/ping/ip=)cat [database-name]) exposed cryptographic MD5 hashes corresponding to specific users.

Step 3: Hash Cracking

The retrieved MD5 hashes were processed using an online decryption platform (CrackStation), revealing plain-text credentials:

- **r00t** : [n100906](#)
- **admin** : [mrsheafy](#)

4. Lateral Movement & Privilege Escalation

4.1 Initial SSH Access

Using the cracked credentials for the user [r00t](#), an interactive secure shell (SSH) session was successfully established:

Bash

ssh r00t@10.114.134.99

Password: n100906

4.2 Local Enumeration

Upon authentication, running the `id` command showed that the `r00t` user belonged to the local `docker` group:

Bash

```
uid=1001(r00t) gid=1001(r00t) groups=1001(r00t),116(docker)
```

4.3 Escape to Root (Docker Breakout)

Because members of the `docker` group can spin up containers with root privileges and mount the host file system, a container escaping technique (documented on GTFOBins) was deployed.

The host root file system (`/`) was mounted into an Alpine Linux container's `/mnt` directory, and a `chroot` environment was executed to drop into a root shell on the host:

Bash

```
docker run -v /:/mnt --rm -it alpine chroot /mnt /bin/sh
```

4.4 Exfiltration

After obtaining the root shell, the operative navigated to the `/root` directory to finalize target objective collection:

1. Checked the `.ssh` directory.
2. Located the private key file (`id_rsa`).
3. Captured the required string (the first 9 letters of the file) for validation.

5. Remediation Recommendations

1. **Sanitize Inputs:** Implement strict input validation or use parameterized functions for the ping API endpoint instead of passing raw user input directly to a system shell interpreter.
2. **Restrict Docker Access:** Limit membership in the `docker` group. Users in this group effectively possess root access due to their ability to run arbitrary containers with host directory mounts.
3. **Password Security:** Enforce robust password complexity policies to prevent simple MD5 hashes from being trivially broken via lookup tables or brute-force tools.